

Parser Failure Atlas

Your parser reads *value flow* well and *control flow* badly. That single split explains almost every error — and it is the worst possible weakness, because control is where security assets live.

1,689 elements 10,947 claims checked 2,464 edges run_011 · VP5 · \$30.23

83.2% Edges that record value flow	9.3% Edges that record control	51% Of elements in a condition have a control edge	49.7% Of the control edges it does write are fully correct	78.3% All claims, fully supported
--	--	--	--	---

First, three corrections to my last version

I had not read the parser prompt. Two of my three top fixes were wrong.

I WAS WRONG — 1

I called the reset/SEQUENCES problem a hallucination. It is a prompt bug. Your parser prompt defines the edge like this:

SEQUENCES — the element is the clock or reset edge of the process assigning the target.

The prompt *tells* the parser that a reset edge is SEQUENCES. The parser obeyed. The verifier's rubric says the opposite. Those 111 "wrong" claims are two prompts disagreeing, not a model inventing things.

I WAS WRONG — 2

I told you to require exact array indices. Do not do that. Your prompt already says the opposite, deliberately:

If a coupling is to one element of an array, name the array itself — never an indexed or sliced expression, and put the range in bits instead.

Targets must be declared names so the asset stage can bind them. My fix would have broken that. The 78 "imprecise target" flags are the verifier not knowing your convention.

I WAS WRONG — 3

The prompts you pasted are not the ones that made this data. Those two short `ANNOTATE` prompts are the v1/v2 versions. They ask the model to use "name, in-source comments, and usage".

Your call goes to `parse_v3.annotate_all_parallel`, which uses `PARSE_ELEMENTS_V3_SYSTEM`. That prompt is far more advanced, strips comments, and forbids name inference. Fix that one.

What survived: the headline numbers, the module ranking, the PMP case, and the finding that the parser drops conditions. What changed is *why*, and what to do about it.

The pattern, in one table

This is the answer to "is there a pattern to where it misses"

Yes, and it is sharp. Sort the edge types by how often the RTL fully supports them:

Two units, kept apart. The parse holds **2,464 relationship entries**. One entry can name several targets, so those expand to **3,734 (type, target) pairs** — and the verifier checks one pair at a time. The "Checked" column below counts pairs. Share-of-parse figures count entries.

EDGE TYPE	CHECKED	FULLY RIGHT	INCOMPLETE	CONTRADICTED	KIND
AGGREGATES	244	99.6%	0.4%	0.0%	value
EXPORTS	430	87.0%	0.2%	6.3%	value
DERIVES_FROM	396	82.8%	16.9%	0.3%	value
CARRIES	480	76.9%	22.3%	0.0%	value
SOURCES	842	70.4%	25.4%	1.9%	value
CAPTURES	326	51.2%	42.0%	5.2%	value + timing
GATES	336	49.7%	48.2%	1.8%	control
SEQUENCES	349	42.1%	26.1%	31.8%	timing

Pure value flow sits at 70–100%. Anything involving a condition or a clock edge falls to 42–51%.

The reason is structural. To record `A SOURCES B` you read one assignment. Everything you need is on that line. To record `A GATES B` you must look at the *branch you are inside* — and nothing in the assignment in front of you points back at it.

Your prompt knows this. It says so, in STEP 2b, in the strongest terms it uses anywhere:

FROM YOUR PROMPT – STEP 2B

"A condition governs an element without ever appearing in that element's value... It is the coupling most often left out, and leaving it out is the single largest loss this task can suffer."

The instruction is right. It is not landing.

The double loss

Control couplings are lost twice over.

- **Half are never written down.** Your own `governing_coverage()` measures it: **173 of 338** elements that appear in a condition carry a control edge. That is 51%.
- **Half of the ones written down are incomplete.** Of 336 GATES checks, 48.2% came back partially supported – the edge exists but the condition is not fully stated.

Multiply those and roughly **one quarter** of the control structure in your design survives into the parse. Value flow survives at four times that rate.

Three failure types, corrected

What is missed, what is misattributed, and what is not the parser's fault

MISSED**The prose knows. The edge does not.**

This is the real failure, and it is worse than "the parser did not notice". Often **it noticed and still did not record it in the structured field**.

Here is the clearest case in your whole corpus. `ctrl_i.csr_we` is the CSR write enable inside PMP. It decides whether anyone can write the memory-permission registers at all.

```
-- RTL, neorv32_cpu_pmp.vhd
L113: if (ctrl_i.csr_addr(11 downto 2) = csr_pmpcfg0_c(...)) and (ctrl_i.csr_we = '1') then
L157: if (ctrl_i.csr_addr(11 downto 4) = csr_pmpaddr0_c(...)) and (ctrl_i.csr_we = '1') then
```

PARSER ROLE – CORRECT

"write enable input", "gates CSR write strobes"

PARSER RELATIONSHIP – THE LOSS

DERIVES_FROM `pmpcfg_we`

DERIVES_FROM `pmpaddr_we`

No GATES edge at all.

The word "gates" is right there in the role. But `role` is free prose. The asset stage reasons over `relationship`, and in that field the gating vanished – replaced by two value-flow edges pointing the wrong way.

In PMP alone, **7 elements** appear in a condition with no control edge – `ctrl_i`, `ctrl_i.lsu_priv` (the privilege level of the access), `ctrl_i.csr_we`, `addr_ls_i`, `pmpaddr`, `addr_mask` and `fail`. (A regex finds 9; `clk_i` and `rstn_i` are excluded because a clock and a reset should not carry a control edge.)

Why the prompt does not prevent it

Two reasons, both structural.

One: there is nowhere to put the condition. Your relationship schema is:

```
{"type": "<EDGE>", "targets": ["<name>"], "bits": "<optional>"}
```

There is no `condition` field. `GATES` can say *that* A governs B. It cannot say *when*. So "only when the FSM is idle and the lock bit is clear" has to go in `functionality` – which is capped at 45 words and already has three other jobs.

Two: STEP 2b asks for a second pass over working notes the model never has to show you. The occurrence profile is internal. Nothing in the output proves the sweep happened, so nothing catches it when it does not.

FIX – HIGHEST VALUE

Add a `condition` field to the relationship schema. Make it required for GATES, SELECTS, CONSTRAINS and OVERRIDES. Let it hold a short quote of the guard, like `"csr_we='1' and cfg_l='0'"`.

This does three things at once: it gives the condition a home, it makes the STEP 2b sweep visible in the output, and it makes the omission checkable — you can count edges with an empty condition.

NOT THE PARSER

Reset vs sequencing is a vocabulary conflict

111 contradicted claims say `rstn_i SEQUENCES x`. I previously called this the parser's biggest hallucination. It is not a hallucination.

FROM YOUR PROMPT – THE EDGE LIST

SEQUENCES – the element is the clock **or reset edge** of the process assigning the target.

The parser did exactly what it was told. The verifier was told something different – that a reset override is not sequencing. Both are defensible readings. They just are not the same reading.

```
-- both prompts are looking at this
if (rstn_i = '0') then
    state <= S_IDLE;           reset FORCES the value
elsif rising_edge(clk_i) then
    state <= state_nxt;       clock ADVANCES it
end if;
```

Your edge vocabulary has no way to tell these apart. `SEQUENCES` covers both. There is a `RESET_BRANCH` site in STEP 2, but no matching *edge* – so the observation is made and then has nowhere to go.

`OVERRIDES` would fit perfectly: *"selects an alternative value for the target in place of the one normally computed"*. That is exactly what a reset does. It is used just 11 times in the whole corpus.

FIX – CHEAPEST BIG WIN

Change one line in the edge list: `SEQUENCES – the element is the clock edge of the process assigning the target.` Drop "or reset edge".

Then add: *"A reset that forces a value in a reset branch is OVERRIDES, not SEQUENCES."*

This removes 111 of 195 contradictions and makes reset behaviour visible as its own thing – which matters, because a reset that clears a lock bit is a security path.

MISATTRIBUTED

Direction and self-reference

After removing the two above, a real misattribution group remains – smaller than I said, but genuine.

EXPORTS on an input – 27 cases

Your prompt says: `EXPORTS – the element reaches the entity boundary. Empty target list.`

An input port *does* reach the boundary. The definition never says which way. So the parser tagged inputs as exports.

FIX

`EXPORTS – the element is driven inside this entity and leaves through an output port. Inputs are never EXPORTS.`

ISOLATED on a used signal – 6 cases

Here the parser broke a rule the prompt states clearly: *"An element whose fields are named individually is coupled through them and is not isolated."* Small, and already covered.

Self-referencing edges

Look at what `pmpcfg` claims: `GATES pmpcfg`. An element gating itself.

That is actually the lock bit – `pmpcfg(i)(cfg_l_c)` blocks writes to `pmpcfg`. Structurally correct. But because targets are de-indexed and there is no `bits` on it, it reads as a meaningless self-loop. The most important security mechanism in the module is recorded in a form nothing downstream can use.

FIX

Require `bits` whenever an edge's source and target are the same element. `pmpcfg GATES pmpcfg bits="7"` is a lock bit. `pmpcfg GATES pmpcfg` is noise.

What makes an element an asset

And how your parser does on each one

An asset is not "an important signal". It is one where an attacker changing it, reading it, or blocking it causes a **security** consequence. A clock is essential and is almost never an asset.

Below are the five behaviours that make an element an asset. Each is paired with how your parser actually handled it. This is the connection you asked for.

ASSET BEHAVIOUR	WHAT THE RTL LOOKS LIKE	PARSER EDGE NEEDED	HOW IT DOES
Holds a value across cycles state is what is worth attacking	<code>x <= y on rising_edge</code>	CAPTURES / kind=register	Good
Carries a value somewhere reach — where a change propagates	<code>b <= a;</code>	SOURCES / CARRIES	Good
Crosses a boundary who outside can touch it	<code>port map / out port</code>	EXPORTS / PORT_MAP	Direction confused
Gates a permission decides if an access happens	<code>if (we='1' and l='0')</code>	GATES	Half missing
Locks other state makes config immutable	<code>if (cfg(L)='0') then write</code>	GATES + bits	Recorded unusably

Read the last column top to bottom. The parser is strong on the two behaviours that describe *data*, and weak on the three that describe *authority*.

That is the worst possible split. Assets are authority elements. A signal that merely carries data is rarely an asset. A signal that decides whether an access is allowed almost always is.

Two examples, side by side

Same module, same call, opposite outcomes

What it got right — a derived comparison

```
-- RTL
L289: cmp_ge(r) <= '1' when TOR_EN else '0';
```

Parser output: `DERIVES_FROM acc_addr`, `DERIVES_FROM pmpaddr`, `SOURCES match`.

All three correct. Everything needed was inside the assignment. This is the parser at its best, and most of your corpus looks like this.

What it lost — the write gate

```
-- RTL
L127: if (pmpcfg_we(i/4) = '1') and (pmpcfg(i)(cfg_l_c) = '0') then
    -- only here can configuration be written
```

That second term is the PMP lock bit. Once set, that region's permissions can never change until reset. It is what stops malicious code unlocking its own sandbox.

The parser recorded `pmpcfg GATES pmpcfg` — technically the lock, but with no bit range and no condition. Nothing downstream can tell that from a self-loop.

And the parser's role for `pmpcfg` reads: *"registered PMP config", "holds region permissions", "gates address writes"*. True, and it sounds like any ordinary write strobe.

Neither of these is a hallucination. The parser saw both. It has a precise vocabulary for the first and no vocabulary at all for the second.

Where it misses, by module

The pattern holds at module level too

MODULE	ELEMENTS	SUPPORTED	WHY
neorv32_imem	27	50.0%	Two memory styles behind a generate guard — pure structural conditioning
neorv32_cpu	115	64.9%	13.2% not established, far above anywhere else
neorv32_cpu_cp_muldiv	70	69.7%	Deep state machine — every assignment sits under a state guard
neorv32_uart	76	71.6%	Many modes and enables
neorv32_bus	870	79.2%	Half the corpus. Mostly repeated routing — little conditioning
neorv32_trng	56	88.1%	Simple dataflow
neorv32_boot_rom	21	92.7%	Almost no branching at all

The ranking is a conditional-logic ranking. Modules with heavy branching sit at the bottom. Modules that mostly move data sit at the top. Same weakness, seen from further away.

The module you piloted on, `neorv32_imem`, is the worst in the corpus. Every early estimate taken from it understated your parser.

What to change, corrected

Ordered by claims recovered per line of prompt edited

#	CHANGE	WHERE IN THE PROMPT	RECOVERS
1	Drop "or reset edge" from SEQUENCES; point reset at OVERRIDES	_EDGES list	~111
2	Add a required <code>condition</code> field for the four governing edges	schema + THE FIELDS	~330
3	Say EXPORTS is outbound only	_EDGES list	~27
4	Require <code>bits</code> on any self-referencing edge	THE FIELDS, targets	—
5	Ask for five observable behaviours by name	`role` spec	—

On change 2 — why a field beats an instruction

STEP 2b already tells the model to sweep for governing sites, in very strong language. It is landing 51% of the time. Telling it harder will not help — your own log records that naming a target population makes the model treat it as a quota.

A required field is different. It is not an instruction to try harder. It is a slot that must be filled, and an empty slot is countable. You can measure it the same way you measure banned words today.

On change 5 — keep the verdict out

Your prompt is right to forbid security words, and the banned-word gate is a good design. Do not weaken it.

Ask instead for *observable facts* that happen to matter for security. Each is checkable in the RTL. None is a judgement.

SUGGESTED ADDITION TO THE ROLE SPEC

Where the profile supports it, also convey:

- whether it makes some other element unwritable while it holds a value
- whether it distinguishes one operating mode from another
- whether it decides that an access completes or is refused
- whether its value is generated rather than computed from inputs
- whether more than one requester competes for it

These map onto the five hooks your prompt already has — HOLDS, GOVERNS, CHOOSES, CROSSES, OUTSIDE — and sharpen the two that carry authority.

How much of this to trust

Read before acting on any number here

The verifier is an LLM too. It was measured by running the same input twice.

- **SUPPORTED** repeats 98% of the time. Solid.
- **PARTIALLY_SUPPORTED** repeats only 67%. Treat the 18.5% as "worth a look".
- Filtering on `confidence >= 0.95` keeps 6,834 claims that repeat 94% of the time.

The most solid findings here do not depend on the verifier at all:

- **51% governing coverage** comes from your own `governing_coverage()`, measured against the RTL by regex.
- **83% value edges vs 9% control edges** is a straight count of the parse output.
- **The SEQUENCES definition** is a line of your prompt. Nothing is inferred.

Two limits. The 51% denominator is a regex lower bound, so true coverage is slightly worse, not better. And these are one verifier run — the reproducibility figures come from repeating a single module, not all seventeen.